

FEMAssembler::buildAffineSystem() 数学模型与实现解读

独立推导说明

2026 年 5 月 27 日

摘要

`FEMAssembler::buildAffineSystem()` 是本工程把频域有限元 `curl-curl` 系统拆成 频率无关 部分的入口, 是 ALPS 快速扫频 (`AlpsSweep`) 以及未来任何 Krylov 矩匹配模型降阶 (MOR) 方法可以直接消费的标准化输入。本文从频域 Maxwell 方程和棱元 Galerkin 离散出发, 独立推导该函数返回的每一项物理含义, 并把推导各步骤精确对回到 `src/fem/FEMAssembler.cpp` 中的代码片段, 给出 `AffineSystem` 各字段的数学定义、装配次序和约束语义。文中只引用本工程公开的 C++ 头文件签名作为接口对照, 所有数学结论均独立给出推导。

目录

1	为什么需要” 仿射” 系统	1
2	从 Maxwell 方程到 <code>curl-curl</code> 弱形式	1
2.1	Galerkin 棱元离散	2
3	$A(\omega)$ 的仿射拆分	2
3.1	无损情形: 单一 M	2
3.2	有损情形: 仿射展开破坏	3
4	端口边界项的秩 1 仿射拆分	3
4.1	端口模的来源: 默认走二维端面本征	3
4.2	从端面本征到端口算子的代数推导	4
4.3	虚拟端口列表	4
5	对应到 <code>buildAffineSystem()</code> 的实现	4
5.1	第一步: 单元参考矩阵 K^T, M^T 缓存	5
5.2	第二步: 稀疏图 + 体积装配 (K, M)	5
5.3	第三步: 损耗标志	5
5.4	第四步: 虚拟端口展开	6
6	PEC 零 Dirichlet 投影	6
7	装配后的不变量	7
8	下游用法: 构造 <code>AlpsSweep</code> 的 ROM 矩阵	7
9	当前限制与扩展点	8

1 为什么需要”仿射”系统

频域电磁有限元在每个频点 ω （角频率， $\omega = 2\pi f$ ）处都需要装配并求解一个大型复对称稀疏线性系统

$$\mathbf{A}(\omega) \mathbf{x}(\omega) = \mathbf{b}(\omega). \quad (1)$$

若每个频点都从零开始走完”算单元矩阵 → 累加全局矩阵 → 因式分解 → 回代”四步，扫频成本随频点数线性增长。把 $\mathbf{A}(\omega)$ 写成

$$\mathbf{A}(\omega) = \sum_q f_q(\omega) \mathbf{A}_q \quad (2)$$

的”仿射展开”（affine decomposition）后，所有 $\mathbf{A}_q$ 都是频率无关的稀疏矩阵，只在 ω 上做轻量加权。这正是 Krylov 矩匹配 MOR、ALPS 自适应 Lanczos–Padé 扫频赖以工作的代数前提。FEMAssembler::buildAffineSystem() 就是一次性产出 (2) 中所有 $\mathbf{A}_q$ 及其加权函数所需要的物理量。

下文先在 §2 推导 $\mathbf{A}(\omega)$ 的连续与离散形式，§3 给出仿射拆分公式，§4 推导端口算子的秩 1 拆分。§5 把这些数学量逐项映射到 C++ 实现，§7–9 讨论实现层的不变量、PEC 投影、损耗判定与目前的限制。

2 从 Maxwell 方程到 curl–curl 弱形式

约定时间因子 $e^{j\omega t}$ 。各向同性线性介质中，频域 Maxwell 方程

$$\nabla \times \mathbf{E} = -j\omega\mu\mathbf{H}, \quad (3)$$

$$\nabla \times \mathbf{H} = (\sigma + j\omega\varepsilon)\mathbf{E} + \mathbf{J}_s \quad (4)$$

消去 $\mathbf{H}$ 得电场 curl–curl 方程

$$\nabla \times (\mu_r^{-1} \nabla \times \mathbf{E}) - k_0^2 \varepsilon_c(\omega) \mathbf{E} = -j\omega\mu_0 \mathbf{J}_s, \quad (5)$$

其中

$$k_0 = \omega\sqrt{\mu_0\varepsilon_0}, \quad \varepsilon_c(\omega) = \varepsilon_r - j\frac{\sigma}{\omega\varepsilon_0}. \quad (6)$$

记计算域为 Ω ，端口面记作 $\Gamma_p \subset \partial\Omega$ ($p = 1, \dots, N_p$)，PEC 面记作 Γ_e 。乘以测试函数 $\mathbf{v} \in H_0(\text{curl}; \Omega)$ （在 Γ_e 上满足 $\mathbf{n} \times \mathbf{v} = 0$ ）并对 Ω 积分，对 curl 项做分部积分，得到弱形式

$$\int_{\Omega} \mu_r^{-1} (\nabla \times \mathbf{v}) \cdot (\nabla \times \mathbf{E}) dV - k_0^2 \int_{\Omega} \varepsilon_c \mathbf{v} \cdot \mathbf{E} dV + \sum_p \int_{\Gamma_p} \mathbf{v} \cdot (\mathbf{n} \times \mu_r^{-1} \nabla \times \mathbf{E}) dS = (\text{右端}). \quad (7)$$

最关键的一步是把 Γ_p 上的边界项代换为只用切向电场表达的端口算子，推导见 §4。

2.1 Galerkin 棱元离散

把 $\mathbf{E}$ 在棱元（Nédélec $H(\text{curl})$ 一阶不完备 / 二阶半完备）基 $\{\mathbf{N}_i\}_{i=1}^N$ 上展开

$$\mathbf{E} \approx \sum_i x_i \mathbf{N}_i, \quad (8)$$

取 $\mathbf{v} = \mathbf{N}_a$, $\mathbf{E}$ 的展开系数为 x_b , 则弱形式 (7) 中的体积积分两项直接给出全局矩阵

$$A_{ab}^{\text{vol}}(\omega) = \underbrace{\int_{\Omega} \mu_r^{-1} (\nabla \times \mathbf{N}_a) \cdot (\nabla \times \mathbf{N}_b) dV}_{\equiv K_{ab}} - k_0^2 \underbrace{\int_{\Omega} \varepsilon_c \mathbf{N}_a \cdot \mathbf{N}_b dV}_{\equiv \tilde{M}_{ab}(\omega)}. \quad (9)$$

注意 K_{ab} 完全频率无关, 而 $\tilde{M}_{ab}(\omega)$ 因含 $\varepsilon_c(\omega)$ 在 $\sigma \neq 0$ 时仍是频率函数。

按单元 T 分解, 标准元素装配

$$K_{ab} = \sum_T \frac{1}{\mu_{r,T}} \mathbf{K}_{ab}^T, \quad \mathbf{K}_{ab}^T = \int_T (\nabla \times \mathbf{N}_a) \cdot (\nabla \times \mathbf{N}_b) dV, \quad (10)$$

$$\tilde{M}_{ab}(\omega) = \sum_T \varepsilon_{c,T}(\omega) \mathbf{M}_{ab}^T, \quad \mathbf{M}_{ab}^T = \int_T \mathbf{N}_a \cdot \mathbf{N}_b dV. \quad (11)$$

$\mathbf{K}^T, \mathbf{M}^T$ 只依赖单元几何与基函数, 不依赖 ω , 是频率无关的单元参考矩阵。

3 $A(\omega)$ 的仿射拆分

3.1 无损情形：单一 M

无损情形 $\sigma \equiv 0$, 因此 $\varepsilon_c = \varepsilon_r$ 也与 ω 无关。体积部分立刻退化为

$$\mathbf{A}^{\text{vol}}(\omega) = \mathbf{K} - k_0^2(\omega) \mathbf{M}, \quad (12)$$

其中

$$\mathbf{K} = \sum_T \frac{1}{\mu_{r,T}} \mathbf{K}^T, \quad \mathbf{M} = \sum_T \varepsilon_{r,T} \mathbf{M}^T. \quad (13)$$

两者都是实对称稀疏矩阵, 与 ω 完全解耦。把端口算子 (§4) 合并进来后得到 lossless 全形仿射展开

$$\boxed{\mathbf{A}(\omega) = \mathbf{K} - k_0^2 \mathbf{M} + \mathbf{j} \sum_{p=1}^{N_v} \beta_p(\omega) \mathbf{m}_p \mathbf{m}_p^T} \quad (14)$$

其中 N_v 是虚拟端口数 (每个物理端口的每个传播模式各一个), $\mathbf{m}_p$ 是端口耦合稀疏向量, $\beta_p(\omega) = \sqrt{\max(0, k_0^2 - k_{c,p}^2)}$ 是该模在频率 ω 处的传播常数。

3.2 有损情形：仿射展开破坏

若存在 $\sigma > 0$, 则 (6) 给出 $\varepsilon_c(\omega) = \varepsilon_r - \mathbf{j}\sigma/(\omega\varepsilon_0)$, (9) 中的质量项变为

$$\tilde{M}_{ab}(\omega) = \sum_T \left(\varepsilon_{r,T} - \mathbf{j} \frac{\sigma_T}{\omega\varepsilon_0} \right) \mathbf{M}_{ab}^T = \varepsilon_r\text{-加权全局} M - \frac{\mathbf{j}}{\omega\varepsilon_0} (\sigma\text{-加权全局} M). \quad (15)$$

这等价于把单一 M 拆成两个全局参考矩阵

$$\mathbf{M} = \sum_T \varepsilon_{r,T} \mathbf{M}^T, \quad \mathbf{M}_{\sigma} = \sum_T \sigma_T \mathbf{M}^T, \quad (16)$$

仿射展开多出一项

$$\mathbf{A}(\omega) = \mathbf{K} - k_0^2 \mathbf{M} + \frac{\mathbf{j}k_0^2}{\omega\varepsilon_0} \mathbf{M}_{\sigma} + \mathbf{j} \sum_p \beta_p(\omega) \mathbf{m}_p \mathbf{m}_p^T. \quad (17)$$

当前实现尚未缓存 $\mathbf{M}_{\sigma}$, 所以遇到 $\sigma > 0$ 时 `AffineSystem::lossless` 被置为 `false`, 调用方 (`AlpsSweep`) 改走逐频点 direct path。具体判定见 §5.3。

4 端口边界项的秩 1 仿射拆分

4.1 端口模的来源：默认走二维端面本征

仿射拆分需要每个端口面 Γ_p 上的”传播模式形状” $\mathbf{e}_{p,m}^\perp(\mathbf{r})$ 和对应截止波数 $k_{c,p,m}^2$ 。本工程现支持三条路径产出这两个量，全部最终落到同一 `PortMode` 数据契约上：

1. **NPM** (`--port-method numerical`, 默认): 把端口面 Γ_p 上的三角剖分单独抽出来当作 2D 网，在其上做 $H(\text{curl})$ 棱元离散，求解二维广义本征值问题

$$\mathbf{K}_{\text{port}} \mathbf{v} = k_c^2 \mathbf{M}_{\text{port}} \mathbf{v}, \quad (18)$$

取最低非伪本征对 $(\mathbf{v}_1, k_{c,1}^2)$ 作为主模。模式形状由 $\mathbf{e}^\perp = \sum_i v_i \mathbf{N}_i^\partial$ 给出，其中 $\mathbf{N}_i^\partial$ 是端口面切向 $H(\text{curl})$ 棱元基。对任何二维截面（矩形 / 脊形 / 圆形 / 偏心）都成立。实现：
`PortModeSolver::solve(faceId) → computeMode → computeMultiMode(faceId, 1)`。

2. **APM** (`--port-method analytic`): 直接套矩形波导 TE10 闭式 $\mathbf{e}_{\text{TE10}} = \sqrt{2/(ab)} \sin(\pi u/a) \hat{v}$, $k_c^2 = (\pi/a)^2$, 再 L2 投影到 FE 端口 dof。仅矩形截面可用。
3. **TFE** (`--port-method tfe`): 与 NPM 同一本征解 (18)，但保留最低 N 个非伪本征对，每个模出一个独立的虚拟端口。

NPM 和 TFE 共用 `PortModeSolver::computeMultiMode`: 单元矩阵对应面 $H(\text{curl})$ 基的 curl-curl 与 mass , 三角形 7 点 Wandzura 求积；本征求解走 LAPACKE_dsygv（无 MKL 时回落 Jacobi 迭代）；零本征对应离散梯度零空间，按相对阈值剔除。

4.2 从端面本征到端口算子的代数推导

设 Γ_p 上传播波导支持 $N_{m,p}$ 个传播或近截止模 $\mathbf{e}_{p,m}^\perp(\mathbf{r})$, $m = 1, \dots, N_{m,p}$ 。无论这些模是来自数值本征 (18) 还是闭式 TE10，对应传播常数都按

$$\beta_{p,m}(\omega) = \sqrt{\max(0, k_0^2 - k_{c,p,m}^2)}. \quad (19)$$

在 Γ_p 上把切向电场展开为

$$\mathbf{n} \times (\mathbf{n} \times \mathbf{E})|_{\Gamma_p} = \sum_{m=1}^{N_{m,p}} \alpha_{p,m} \mathbf{e}_{p,m}^\perp + \mathbf{E}^{\text{inc}}, \quad (20)$$

代入 (7) 中的端口边界积分后，无源端口在弱形式中贡献的部分是

$$\int_{\Gamma_p} \mathbf{v} \cdot \left[\mathbf{j} \sum_m \beta_{p,m} (\mathbf{e}_{p,m}^\perp \cdot \mathbf{E}) \mathbf{e}_{p,m}^\perp \right] dS, \quad (21)$$

等价于在矩阵中加入秩 1 算子

$$\mathbf{j} \sum_m \beta_{p,m} \mathbf{m}_{p,m} \mathbf{m}_{p,m}^\top, \quad [\mathbf{m}_{p,m}]_a = \int_{\Gamma_p} \mathbf{N}_a \cdot \mathbf{e}_{p,m}^\perp dS. \quad (22)$$

$\mathbf{m}_{p,m}$ 在代码中称为 `couplingWeights`, 是稀疏的——只有支撑落在 Γ_p 的边自由度的分量非零。 $\mathbf{m}_{p,m}$ 与 ω 无关； $\beta_{p,m}(\omega)$ 是仿射形式中唯一的频率函数。NPM/TFE 路径下 $\mathbf{m}_{p,m}$ 直接由 $\mathbf{M}_{\text{port}} \mathbf{v}_m$ 给出（ M -内积投影），APM 路径下 $\mathbf{m}_{p,m}$ 是闭式 $\mathbf{e}^\perp$ 在 $H(\text{curl})$ 端口 dof 上的 L2 投影。三者共用同一 `PortMode::couplingWeights` 字段。

4.3 虚拟端口列表

把 (22) 对所有物理端口、所有解析模式平铺成一张虚拟端口 列表, (14) 中的端口和写成

$$j \sum_{v=1}^{N_v} \beta_v(\omega) \mathbf{m}_v \mathbf{m}_v^T, \quad N_v = \sum_p N_{m,p}, \quad (23)$$

其中索引 v 与 (projectPortIndex, modeIndex) 一一对应。单模 NPM/APM 路径下 $N_{m,p} = 1$, $N_v = N_p$ 直接退化。TFE 多模情况下, 每个物理端口可能贡献若干虚拟端口; 最多一个被标记为 isExcitationMode, 承载入射幅相, 其余作纯吸收端口 (在 (7) 中入射项 $\mathbf{E}^{\text{inc}}$ 为零)。

5 对应到 buildAffineSystem() 的实现

接口契约见 include/bpfem/fem/FEMAssembler.hpp 的 AffineSystem:

```
struct AffineSystem {
    SparseMatrix K; // sum_e (1/mu_r,e) * K_e
    SparseMatrix M; // sum_e (eps_r,e) * M_e
    std::vector<std::vector<std::pair<int,double>>> portCoupling; // m_p (per virtual port)
    std::vector<double> portCutoffSquared; // k_{c,p}^2
    std::vector<int> portFaceIds; // physical face id
    std::vector<int> projectPortIndex; // index into project_.ports
    std::vector<bool> isExcitationMode; // true = incident-bearing mode
    std::unordered_set<int> constrainedDofs; // PEC zero-Dirichlet mask
    bool lossless = true; // false if any sigma > 0
};
```

下面分四步把数学量映射到代码 (行号引自 src/fem/FEMAssembler.cpp)。

5.1 第一步: 单元参考矩阵 K^T, M^T 缓存

ensureElementCache() 对每个有效四面体 T 计算一次几何 + 基函数组合

$$(\mathbf{K}_{ab}^T, \mathbf{M}_{ab}^T) = \left(\sum_q w_q V_T (\nabla \times \mathbf{N}_a) \cdot (\nabla \times \mathbf{N}_b), \sum_q w_q V_T \mathbf{N}_a \cdot \mathbf{N}_b \right) \quad (24)$$

其中 $\{(w_q, \lambda_q)\}$ 是 tetraQuadraturePoints() 给出的 11 点四面体重心坐标求积公式。对应代码:

```
for (a = 0..dof) for (b = a..dof)
    for each quadrature point qp:
        curlCurl += qp.weight * volume * dot(rowBasis.curl, colBasis.curl);
        mass += qp.weight * volume * dot(rowBasis.value, colBasis.value);
    entry.curlCurlGeo[idx] = curlCurl; // K^T_{ab}
    entry.massGeo [idx] = mass; // M^T_{ab}
    entry.invMu = 1 / mu_r,T;
    entry.epsR = eps_r,T;
    entry.conductivity = sigma_T; // 用于 lossless 判定
```

K^T, M^T 用上三角紧致索引存储 upperTriIndex(a,b,dof), 保证装配后 K, M 都是 CSR 上三角对称矩阵。

5.2 第二步：稀疏图 + 体积装配 (K, M)

buildAffineSystem() 不调用与 assemble() 共享的稀疏图，而是单独构造一个只含体积耦合的稀疏图：

```
const auto pattern = volumePatternFor(out.constrainedDofs);
out.K = assembleVolumePiece(pattern, out.constrainedDofs, /*useCurlCurl=*/true);
out.M = assembleVolumePiece(pattern, out.constrainedDofs, /*useCurlCurl=*/false);
```

理由：ALPS 的 K, M 永远不会出现端口外积非零位置，剥掉端口图后稀疏图 严格小于等于 单频点装配的图，能把 PARDISO 符号分解成本压到最小。assembleVolumePiece 内对每个单元做

$$K += \frac{1}{\mu_{r,T}} K^T, \quad M += \varepsilon_{r,T} M^T, \quad (25)$$

其中 K^T, M^T 来自第一步缓存， σ_T 项故意不进入 M ，保留实矩阵性质（满足 (13)）。

5.3 第三步：损耗标志

```
out.lossless = true;
for (const auto& entry : elementCache_) {
    if (entry.valid && entry.conductivity > 0.0) {
        out.lossless = false;
        break;
    }
}
```

对应数学事实：单一实对称 M 不再能表达 $\varepsilon_c(\omega)$ 在 $\sigma > 0$ 时的 j/ω 频率依赖，因此 (14) 不再充分。AlpsSweep 读到 lossless==false 时直接抛 runtime_error 提示 --sweep direct。

5.4 第四步：虚拟端口展开

最后由注册的 IBoundaryCondition 列表（典型为 WavePortBC）通过 affineContributions(ctx) 给出每个虚拟端口的 $(\mathbf{m}_v, k_{c,v}^2, \dots)$ 。WavePortBC::affineContributions 内部对每个 ProjectPort 先看 portModeSolver.multiMode(faceId)：

1. TFE 多模 (multiMode != nullptr)：按 modeIndex 顺序把每个解析模 m 都打成一个 AffinePortContribution, isExcitationMode 仅在 $m = \text{excitationModeIndex}$ 且 port.excited 同时成立时为真；
2. NPM/APM 单模 (multiMode == nullptr)：直接调用 portModeSolver.solve(faceId) (NPM 走二维 H(curl) 本征，APM 走闭式 TE10 投影)，产出唯一一个虚拟端口。

对每个 contribution 内部还会过滤掉落在 ctx.constrainedDofs 里的 dof，保证 $\mathbf{m}_v$ 与 K, M 共用同一稀疏图。汇总循环是：

```
for (const auto& cond : bcs_) {
    if (!cond) continue;
    for (auto& contrib : cond->affineContributions(ctx)) {
        out.portFaceIds.push_back (contrib.faceId);
        out.portCutoffSquared.push_back(contrib.cutoffSquared);
        out.portCoupling.push_back (std::move(contrib.coupling));
        out.projectPortIndex.push_back(contrib.projectPortIndex);
        out.isExcitationMode.push_back(contrib.isExcitationMode);
    }
}
```

把这四步串起来, `buildAffineSystem()` 实际给出的就是 (14) 所需要的全部素材, 只是把抽象数学量映射到 C++ 数据结构:

数学量	代码字段	装配位置
$\mathbf{K} = \sum_T \mu_{r,T}^{-1} \mathbf{K}^T$	<code>out.K</code>	§5.2 <code>assembleVolumePiece(..., true)</code>
$\mathbf{M} = \sum_T \varepsilon_{r,T} \mathbf{M}^T$	<code>out.M</code>	§5.2 <code>assembleVolumePiece(..., false)</code>
$\mathbf{m}_v$ (每虚拟端口)	<code>out.portCoupling[v]</code>	§5.4 <code>affineContributions</code>
$k_{c,v}^2$	<code>out.portCutoffSquared[v]</code>	同上
$v \rightarrow$ 物理端口面 id	<code>out.portFaceIds[v]</code>	同上
$v \rightarrow$ ProjectPort 索引	<code>out.projectPortIndex[v]</code>	同上
是否激励模	<code>out.isExcitationMode[v]</code>	同上
PEC 约束 dof 集	<code>out.constrainedDofs</code>	§6 <code>cachedConstrainedDofs</code>
$\sigma \equiv 0$ 判定	<code>out.lossless</code>	§5.3

表 1: `AffineSystem` 字段与数学模型量的一一对应。

6 PEC 零 Dirichlet 投影

PEC 面 Γ_e 上要求 $\mathbf{n} \times \mathbf{E} = 0$, 棱元离散后即“ Γ_e 上所有切向边 dof 强置零”。代码用 `collectPecConstrainedDofs()` 扫一次表面三角列表, 把不属于任何端口的边全部塞进 `cachedConstrainedDofs`。这个集合在 `buildAffineSystem()` 里有三处用途:

1. **稀疏图:** `volumePatternFor` 中跳过任意端含约束 dof 的 (a, b) 对, 保证 $\mathbf{K}, \mathbf{M}$ 不在这些行/列上插入条目。
2. **累加:** `assembleVolumePiece` 中重复同样的过滤, 单元贡献不写入约束行/列。
3. **端口耦合:** `WavePortBC::affineContributions` 中对每个 $\mathbf{m}_v$ 内部过滤约束 dof, 避免端口耦合向量的支撑超出 $\mathbf{K}, \mathbf{M}$ 的稀疏图。

注意 `buildAffineSystem()` 不在 $\mathbf{K}, \mathbf{M}$ 上做 `imposeZeroDirichlet` 那样的“主对角线置 1”操作; 它选择的是投影路径——直接把约束 dof 从矩阵稀疏图上剔除, 等价于在缩减子空间 $\{x_i = 0, i \in \text{PEC}\}$ 上写下系统。约束 dof 集仍随 `AffineSystem` 传给 `AlpsSweep`, 方便下游做一致性检查或重建满空间向量。

7 装配后的不变量

1. **$\mathbf{K}, \mathbf{M}$ 实对称:** 上三角 CSR 复矩阵但虚部恒为 0, 由 (13) 本身的实数性保证; 引入张量介质后需要扩展为多分量缓存。
2. **虚拟端口排序:** `out.portCoupling` 等数组按“`projectPortIndex` 升序、其内 `modeIndex` 升序”排列, 与 `WavePortBC::affineContributions` 的循环次序一致。`AlpsSweep` 中的 `dominantVirtualByProject[]` 依赖这个顺序定位每个物理端口的主模虚拟端口。
3. **激励唯一性:** 每个 `projectPortIndex` 至多一个 `isExcitationMode==true`。NPM/APM 单模情形下“唯一”等价于“全部”。
4. **$\mathbf{m}_v$ 与 $\mathbf{K}, \mathbf{M}$ 同图:** 约束 dof 在三处一致剔除, 不会出现 $\mathbf{m}_v \mathbf{m}_v^T$ 在 $\mathbf{K} + j\beta_v \mathbf{m}_v \mathbf{m}_v^T$ 引入新的非零位置。
5. **复对称投影:** 下游 ALPS 用不带共轭的双线性 $\mathbf{V}^T \mathbf{K} \mathbf{V}$ 做 Galerkin 投影。 $\mathbf{K}, \mathbf{M}$ 的实对称性 + $\mathbf{m}_v \mathbf{m}_v^T$ 的复对称性共同保证 $\mathbf{A}(\omega)$ 在 (14) 下严格复对称, 进而 ROM 也保留这一结

构。

8 下游用法：构造 AlpsSweep 的 ROM 矩阵

为完整说明 `buildAffineSystem()` 输出的”够用性”，简述下游 `AlpsSweep::buildOffline` 对它的消费方式。在中心频率 ω_0 处，`AlpsSweep` 以

$$\mathbf{A}_0 = \mathbf{K} - k_0^2(\omega_0)\mathbf{M} + j \sum_v \beta_v(\omega_0) \mathbf{m}_v \mathbf{m}_v^T \quad (26)$$

做 Pardiso 因式分解；接着对每个虚拟端口求解 $\mathbf{A}_0 \mathbf{w}_v^{(1)} = \mathbf{m}_v$ 作为 Krylov 子空间起始向量；递推

$$\mathbf{A}_0 \mathbf{w}_v^{(k+1)} = \mathbf{M} \mathbf{w}_v^{(k)} \quad (27)$$

(注意右端只用到频率无关的 $\mathbf{M}$) 生成 r 阶矩匹配子空间，再做 modified Gram-Schmidt 正交化得到 $\mathbf{V}$ 。投影后的 ROM

$$\mathbf{K}_r = \mathbf{V}^T \mathbf{K} \mathbf{V}, \quad \mathbf{M}_r = \mathbf{V}^T \mathbf{M} \mathbf{V}, \quad \mathbf{m}_{v,r} = \mathbf{V}^T \mathbf{m}_v \quad (28)$$

是几十到几百维的稠密小矩阵，最终在每个频点 ω 求解

$$\left(\mathbf{K}_r - k_0^2 \mathbf{M}_r + j \sum_v \beta_v(\omega) \mathbf{m}_{v,r} \mathbf{m}_{v,r}^T \right) \mathbf{x}_r(\omega) = \mathbf{b}_r(\omega) \quad (29)$$

即可获得端口响应，避免重建全空间场。整条链路对 $\mathbf{K}, \mathbf{M}, \mathbf{m}_v, k_{c,v}^2$ 的需求都由 `buildAffineSystem()` 一次性满足。

9 当前限制与扩展点

1. **有损介质未仿射化**: $\sigma > 0$ 时 `lossless==false`，调用方走 direct path。延伸方向是缓存 $\mathbf{M}_\sigma = \sum_T \sigma_T \mathbf{M}^T$ 作为第三块全局矩阵，仿射展开多一项 $j(k_0^2/(\omega\epsilon_0))\mathbf{M}_\sigma$ ，ALPS Krylov 递推再加一个右端 $\mathbf{M}_\sigma \mathbf{w}_v^{(k)}$ 。
2. **各向异性介质**: (10)–(11) 假设 μ_r, ϵ_r 标量，对张量介质需要把 $\mathbf{K}^T, \mathbf{M}^T$ 拆成多分量并各自加权。
3. **端口模式频率无关性**: (22) 假设 $\mathbf{e}_{p,m}^\perp$ 与 ω 无关，这是 ALPS 仿射展开的代数前提。NPM/TFE 路径下这要求二维 H(curl) 本征问题 (18) 的特征对（在频带内不发生模式交叉的前提下）频率无关，对常规均匀填充波导自动满足；色散填充或开放波导需要单独处理。APM 路径假设矩形 TE10 闭式同样频率无关。
4. **ABC/SIBC 边界条件**: 当前 `IBoundaryCondition::affineContributions` 默认空。吸收边界、阻抗边界一般给出多项形式（含 $\sqrt{j\omega}$ 等），若要纳入仿射展开需要额外的项。

10 小结

`FEMAssembler::buildAffineSystem()` 把频域 curl-curl FEM 系统在 lossless 假设下彻底拆成 (14) 的”频率无关矩阵 + 频率函数加权”形式，为 ALPS 等矩匹配快速扫频提供严格的代数基础。其实现遵循三条主线：单元参考矩阵缓存 (§5.1)、体积全局矩阵装配 (§5.2)、经 `IBoundaryCondition` 分发的虚拟端口展开 (§5.4)；PEC 投影 (§6) 和损耗标志 (§5.3) 作为两条横切机制保证后续 ROM 与单频点 direct path 的等价性。